Entity Framework, but you could consider many other alternatives, such as Dapper and NHibernate, each of which has its own set of tradeoffs.

12.1.3 When should you choose EF Core?

Microsoft designed EF Core as a reimagining of the mature Entity Framework 6.x (EF 6.x) ORM, which it released in 2008. With many years of development behind it, EF 6.x was a stable and feature-rich ORM, but it's no longer under active development.

EF Core, released in 2016, is a comparatively new project. The APIs of EF Core are designed to be close to those of EF 6.x—though they aren't identical—but the core components have been completely rewritten. You should consider EF Core to be distinct from EF 6.x; upgrading directly from EF 6.x to EF Core is nontrivial.

Although Microsoft supports both EF Core and EF 6.x, EF 6.x isn't recommended for new .NET applications. There's little reason to start a new application with EF 6.x these days, but the exact tradeoffs will depend largely on your specific app. If you decide to choose EF 6.x instead of EF Core, make sure that you understand what you're sacrificing. Also make sure that you keep an eye on the guidance and feature comparison from the EF team at http://mng.bz/GxgA.

If you decide to use an ORM for your app, EF Core is a great contender. It's also supported out of the box by various other subsystems of ASP.NET Core. *In* chapter 23 you'll see how to use EF Core with the ASP.NET Core Identity authentication system for managing users in your apps.

Before I get into the nitty-gritty of using EF Core in your app, I'll describe the application we're going to be using as the case study for this chapter. I'll